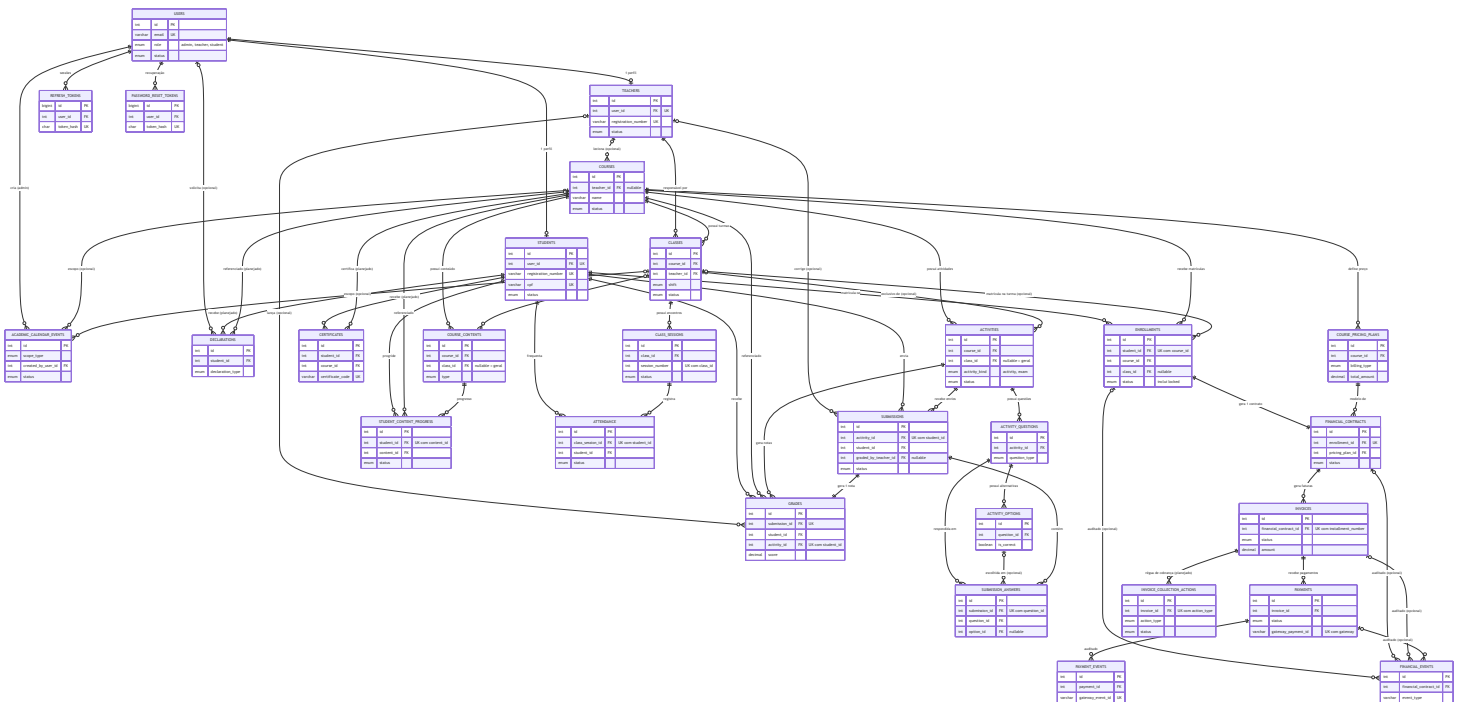


Diagrama — Modelo Entidade-Relacionamento do Banco

Gerado a partir do `information_schema` do banco `coursehub_escola` em 2026-08-02 (28 tabelas, pós-remoção de `activity_submissions` / `staff`). Ver `docs/database/relational-model.md` para a descrição completa coluna a coluna — este diagrama mostra apenas colunas-chave e de negócio central, para manter a leitura viável.



Que problema este diagrama esclarece

Antes deste documento, entender como as 28 tabelas do CourseHub se conectam exigia ler `backend/db.js`, cada `service`, e inferir relações a partir do SQL espalhado por `services/**`. Este ERD é a primeira representação visual única e verificada do schema completo — permite responder em segundos perguntas como "o que acontece com as faturas se eu apagar um contrato financeiro?" ou "de onde vem a nota de um aluno?" sem precisar ler código.

Como interpretar os elementos

- Cardinalidade nas linhas:** `||` = exatamente um; `o|` = zero ou um (opcional, lado 1); `o{` = zero ou muitos (lado N, opcional). Por exemplo, `TEACHERS |o--o{ COURSES` significa que um curso pode

ter zero ou um professor (não muitos), e um professor pode lecionar zero ou muitos cursos — reflete `courses.teacher_id nullable`.

- **Rótulos "(opcional)"** nas relações indicam FKs nullable no schema real — não são uma escolha estética, são a regra de negócio (ex. `class_id nullable` em atividades/conteúdo = "geral vs. exclusivo de turma", documentado em `business-rules.md` seção 3).
- **Rótulos "(planejado)"**: `CERTIFICATES`, `DECLARATIONS` e `INVOICE_COLLECTION_ACTIONS` aparecem no diagrama porque existem no schema, mas nenhuma delas tem código de aplicação por trás — estão marcadas explicitamente para que ninguém as leia como funcionalidade ativa.
- **UK nos atributos**: marca colunas ou pares de colunas com restrição `UNIQUE` — muitas delas carregam regra de negócio (ex. `ENROLLMENTS.student_id "UK com course_id"` = um aluno não pode ter duas matrículas ativas no mesmo curso).

Que decisões arquiteturais este diagrama evidencia

- **Perfis satélites em vez de herança de tabela única**: `users` guarda só o que é comum a todo mundo (autenticação); `students` / `teachers` guardam o que é específico de cada papel, cada um em 1:1 com `users`. Isso evita uma tabela `users` com dezenas de colunas nullable dependendo do papel.
- **O par `submissions` + `grades`, separado mas 1:1**: um envio de aluno e a nota que ele recebe são conceitos distintos (o envio existe antes de ser corrigido), mas a restrição `UNIQUE` em `grades.submission_id` garante que nunca existam duas notas para o mesmo envio.
- **Auditoria financeira em duas trilhas paralelas**: `payment_events` (granular, por pagamento) e `financial_events` (amplo, qualquer entidade financeira) coexistem porque respondem perguntas diferentes — "o que aconteceu com este pagamento específico?" vs. "o que um admin alterou neste contrato?".

Trade-offs que este modelo representa

- **Duplicação de dados em `students` / `teachers`** (nome, e-mail, gênero também existem em `users`): favorece leitura simples (sem join) em troca do risco de os dois ficarem dessincronizados se um dia alguém atualizar só um lado — hoje mitigado porque `services/profile/profileService.js` sempre atualiza as duas tabelas na mesma transação.
- **Sem exclusão física em nenhuma entidade central**: garante histórico completo (acadêmico e financeiro nunca desaparece), em troca de crescimento permanente das tabelas e da obrigação de toda consulta de listagem filtrar por `status` explicitamente.
- **Réguas de negócio complexas modeladas antes de implementadas** (`invoice_collection_actions`): o schema já reflete o desenho final pretendido do fluxo de

cobrança, mesmo sem o motor que o executa — bom para planejamento, mas exige disciplina para não presumir que a funcionalidade existe só porque a tabela existe.

Como este diagrama deve evoluir

Ao adicionar uma nova entidade central (por exemplo, se `certificates` ganhar implementação real), atualize este ERD e `relational-model.md` na mesma revisão que o código — os dois documentos devem sempre refletir o `schema information_schema` real, nunca um `schema` aspiracional. Se uma nova tabela reintroduzir o conceito de papéis adicionais (`manager` / `staff` , removidos em 2026-08-02), documente explicitamente por que a decisão anterior foi revertida.